Apprentissage Auto-Supervisé et Test-Time Training

SimCLR + Vision Transformer + CIFAR-10-C

Rapport de TER – Master 1 Vision et Machine Intelligence

Maksym DOLHOV et Fallou DIOUF

Année universitaire 2025–2026

Table des matières

Résumé **3**

1 Introduction **4**

2 Contexte théorique **6**
2.1 Apprentissage auto-supervisé 6
2.2 SimCLR 6
2.3 Vision Transformers 7
2.4 Linear Probe et Fine-Tuning 7
2.5 Test-Time Training et TENT 8

3 Méthodologie **9**
3.1 Pipeline global 9
3.2 Données : CIFAR-10 et CIFAR-10-C 10
3.3 Architecture du modèle 10
3.4 Stage A : Pré-entraînement SimCLR 10
3.5 Stage B.1 : Linear Probe 11
3.6 Stage B.2 : Fine-tuning 11
3.7 Stage C : Test-Time Training 11

4 Implémentation **12**
4.1 Structure générale du code 12
4.2 BaseTrainer 12
4.3 SimCLRTrainer 13
4.4 LinearProbeTrainer 13
4.5 FineTuneTrainer 13
4.6 TestTimeAdapter (TENT) 14
4.7 Notebook Colab et reproductibilité 14

5 Résultats **15**
5.1 Convergence SimCLR 15
5.2 Linear Probe 15
5.3 Fine-tuning 15
5.4 CIFAR-10-C : baseline vs TTT 15
5.5 Analyse des gains de robustesse 15

6 Discussion **16**

7 Conclusion et perspectives **18**

A Annexes **21**
A.1 Structure du dépôt 21
A.2 Configuration YAML 21
A.3 Extraits de code essentiels 22
A.4 Environnement d'exécution 22
A.5 Commandes d'exécution du pipeline 23
A.6 Résultats TensorBoard 23
A.7 Reproductibilité 23

Résumé

Ce travail s'inscrit dans le cadre du TER du second semestre du **Master 1 Vision et Machine Intelligence (M1 VMI)**. Il porte sur l'étude de la robustesse des modèles de vision face aux décalages de distribution, en combinant apprentissage auto-supervisé, évaluation supervisée et adaptation en ligne au moment du test.

Nous proposons un pipeline complet reposant sur un Vision Transformer (ViT) pré-entraîné avec la méthode contrastive SimCLR, puis évalué sur la tâche de classification CIFAR-10 via *linear probe* et *fine-tuning*. Enfin, nous appliquons la méthode TENT (Test-Time Entropy Minimization) afin d'améliorer la robustesse du modèle sur le benchmark CIFAR-10-C, qui regroupe quinze types de corruptions visuelles à cinq niveaux de sévérité.

Le pipeline développé est modulaire, reproductible et entièrement exécutable sur Google Colab. Il intègre la gestion des checkpoints, une journalisation avancée, ainsi que l'exécution contrôlée des différents stages. Les résultats montrent que le pré-entraînement auto-supervisé permet d'obtenir des représentations de haute qualité, conduisant à une précision supérieure à 90% après fine-tuning. Sur CIFAR-10-C, l'adaptation en ligne via TENT améliore systématiquement la robustesse du modèle, avec des gains significatifs sur la majorité des corruptions.

Ce rapport présente les fondements théoriques des méthodes utilisées, détaille l'architecture du pipeline, analyse les performances obtenues et discute les perspectives offertes par l'apprentissage auto-supervisé et le test-time training pour la robustesse des modèles de vision.

Chapitre 1

Introduction

Les modèles de vision par ordinateur ont connu des progrès considérables au cours de la dernière décennie, notamment grâce à l'essor de l'apprentissage profond. Cependant, ces avancées reposent souvent sur la disponibilité de grandes quantités de données annotées, dont la collecte est coûteuse, longue et parfois impraticable. Dans ce contexte, l'apprentissage auto-supervisé (Self-Supervised Learning, SSL) s'est imposé comme une alternative prometteuse, permettant d'exploiter efficacement des données non annotées pour apprendre des représentations visuelles robustes.

Parmi les méthodes SSL les plus influentes, SimCLR [**?**] a démontré qu'un simple objectif de contraste, associé à des augmentations visuelles fortes, permet d'obtenir des représentations de haute qualité, compétitives avec des approches supervisées. Parallèlement, l'introduction des Vision Transformers (ViT) [**?**] a profondément modifié le paysage de la vision artificielle, en remplaçant les convolutions traditionnelles par des mécanismes d'attention globale. L'association de SimCLR et de ViT constitue ainsi une combinaison particulièrement puissante pour l'apprentissage de représentations visuelles.

Cependant, même des modèles bien entraînés restent sensibles aux décalages de distribution (domain shift), notamment lorsqu'ils sont confrontés à des corruptions ou perturbations non vues durant l'entraînement. Ce problème est particulièrement visible sur des benchmarks tels que CIFAR-10-C [**?**], qui évaluent la robustesse des modèles face à des corruptions réalistes (bruit, flou, météo, compression, etc.). Pour répondre à cette problématique, des méthodes d'adaptation en ligne, comme le Test-Time Training (TTT) et en particulier TENT [**?**], proposent d'adapter le modèle directement au moment du test, sans accès aux labels, en minimisant l'entropie des prédictions.

Ce travail s'inscrit dans cette dynamique et propose un pipeline complet combinant :
- un pré-entraînement auto-supervisé SimCLR sur un Vision Transformer,
- une évaluation supervisée via *linear probe* et *fine-tuning*,
- une adaptation en ligne par TTT pour améliorer la robustesse aux corruptions.

L'objectif principal est d'étudier l'impact de l'apprentissage auto-supervisé sur la robustesse du modèle, ainsi que les gains apportés par l'adaptation en ligne sur CIFAR-10-C. Pour cela, nous avons développé un pipeline modulaire, reproductible et entièrement exécutable sur Google Colab, incluant gestion des checkpoints, logging avancé, et exécution

contrôlée des différents stages.

Ce rapport présente d'abord les fondements théoriques des méthodes utilisées, puis détaille la méthodologie et l'implémentation du pipeline. Enfin, nous analysons les résultats obtenus sur CIFAR-10 et CIFAR-10-C, avant de conclure sur les perspectives offertes par l'apprentissage auto-supervisé et le test-time training.

Chapitre 2

Contexte théorique

Ce chapitre présente les fondements conceptuels nécessaires à la compréhension du pipeline développé dans ce travail. Nous introduisons d'abord les principes de l'apprentissage auto-supervisé, puis nous détaillons la méthode SimCLR utilisée pour le pré-entraînement. Nous présentons ensuite les Vision Transformers, qui constituent l'architecture du modèle étudié. Enfin, nous décrivons les approches supervisées d'évaluation (linear probe et fine-tuning), ainsi que les méthodes d'adaptation en ligne au moment du test, en particulier TENT.

2.1 Apprentissage auto-supervisé

L'apprentissage auto-supervisé (Self-Supervised Learning, SSL) est un paradigme d'apprentissage dans lequel un modèle apprend à partir de données non annotées en construisant lui-même la supervision nécessaire à l'entraînement. Contrairement à l'apprentissage supervisé, qui repose sur des labels coûteux à collecter, le SSL exploite des tâches prétextes permettant d'extraire des représentations visuelles robustes.

Dans le domaine de la vision, les tâches prétextes incluent notamment :

— la prédiction de transformations appliquées à une image,
— la reconstitution de régions masquées,
— la discrimination entre différentes vues d'un même échantillon,
— l'ordonnancement temporel ou spatial.

Les méthodes modernes de SSL se divisent en deux grandes familles :

— **méthodes contrastives**, qui rapprochent les représentations de vues positives et éloignent celles de vues négatives ;
— **méthodes non-contrastives**, qui reposent sur des mécanismes d'alignement et de régularisation sans négatifs explicites.

SimCLR, utilisé dans ce projet, appartient à la première famille et constitue l'une des approches contrastives les plus influentes.

2.2 SimCLR

SimCLR [**?**] est une méthode d'apprentissage contrastif reposant sur trois éléments clés :

— des augmentations visuelles fortes (crop, couleur, flou, etc.) ;
— un encodeur (CNN ou ViT) produisant des représentations ;

— une tête de projection non linéaire optimisée avec la perte InfoNCE.

Le principe est le suivant : pour chaque image, deux augmentations indépendantes sont générées, formant une paire positive. Le modèle doit rapprocher leurs représentations tout en éloignant celles des autres images du batch (paires négatives). L'objectif InfoNCE maximise la similarité des vues positives et minimise celle des vues négatives.

SimCLR a démontré qu'un entraînement contrastif sur de grandes quantités de données non annotées permet d'obtenir des représentations transférables très performantes, souvent comparables à celles obtenues par supervision.

2.3 Vision Transformers

Les Vision Transformers (ViT) [?] transposent les mécanismes d'attention des Transformers au domaine de la vision. Contrairement aux réseaux convolutifs, les ViT opèrent sur des patchs d'image, linéarisés puis projetés dans un espace de dimension fixe.

Un ViT se compose de :

— une découpe de l'image en patchs réguliers,
— une projection linéaire des patchs,
— une position absolue ajoutée aux embeddings,
— une succession de blocs Transformer (self-attention + MLP),
— une tête de classification.

Les ViT présentent plusieurs avantages :

— une capacité d'attention globale,
— une meilleure scalabilité,
— une compatibilité naturelle avec les méthodes SSL.

Dans ce projet, nous utilisons un ViT-Tiny, adapté aux images de petite résolution comme CIFAR-10.

2.4 Linear Probe et Fine-Tuning

Après le pré-entraînement auto-supervisé, deux stratégies sont couramment utilisées pour évaluer la qualité des représentations apprises :

Linear Probe

Le linear probe consiste à geler l'encodeur pré-entraîné et à entraîner uniquement une couche linéaire sur les représentations. Cette méthode permet d'évaluer la qualité intrinsèque des représentations sans modifier l'encodeur.

Fine-Tuning

Le fine-tuning consiste à réentraîner l'ensemble du modèle (ou une partie) sur la tâche cible. Cette approche permet d'adapter finement les représentations au dataset supervisé et conduit généralement à de meilleures performances.

Dans ce projet, nous utilisons les deux approches afin de mesurer l'impact du pré-entraînement SimCLR.

2.5 Test-Time Training et TENT

Les modèles de vision sont souvent sensibles aux décalages de distribution, notamment lorsqu'ils sont confrontés à des corruptions non vues durant l'entraînement. Le benchmark CIFAR-10-C [**?**] permet d'évaluer cette robustesse en appliquant quinze types de corruptions à cinq niveaux de sévérité.

Le Test-Time Training (TTT) vise à adapter le modèle directement au moment du test, sans labels. Parmi les méthodes existantes, TENT (Test-Time Entropy Minimization) [**?**] est l'une des plus simples et efficaces. Elle consiste à :

— geler toutes les couches sauf les normalisations,

— minimiser l'entropie des prédictions sur les données corrompues,

— adapter le modèle en ligne, batch par batch.

Cette adaptation locale permet d'améliorer la robustesse du modèle sans réentraîner l'ensemble du réseau ni accéder aux labels.

Chapitre 3

Méthodologie

Ce chapitre décrit en détail la méthodologie adoptée pour concevoir, entraîner et évaluer le modèle étudié. Nous présentons d'abord l'organisation générale du pipeline, puis les jeux de données utilisés, l'architecture du modèle, et enfin les quatre stages successifs : pré-entraînement auto-supervisé, linear probe, fine-tuning et adaptation en ligne au moment du test.

3.1 Pipeline global

Le pipeline développé dans ce projet suit une structure modulaire composée de quatre stages indépendants mais complémentaires. Chaque stage peut être exécuté séparément grâce à un système de drapeaux (`RUN_STAGE_A`, `RUN_STAGE_B1`, etc.), permettant de reprendre une expérience sans réexécuter l'ensemble du processus. Le pipeline est organisé comme suit :

1. **Stage A : Pré-entraînement SimCLR** Apprentissage auto-supervisé du Vision Transformer sur CIFAR-10 sans labels.
2. **Stage B.1 : Linear Probe** Évaluation de la qualité des représentations apprises en entraînant uniquement une couche linéaire.
3. **Stage B.2 : Fine-tuning** Réentraînement complet du modèle sur CIFAR-10 avec labels.
4. **Stage C : Test-Time Training (TENT)** Adaptation en ligne du modèle sur CIFAR-10-C pour améliorer la robustesse.

Le pipeline inclut également :
— une gestion automatique des checkpoints,
— un système de logs détaillés (CSV, TensorBoard),
— une compatibilité complète avec Google Drive pour la persistance,
— une configuration centralisée via un fichier YAML.
Cette organisation garantit la reproductibilité, la flexibilité et la facilité d'expérimentation.

3.2 Données : CIFAR-10 et CIFAR-10-C

CIFAR-10

CIFAR-10 est un jeu de données composé de 60 000 images couleur de taille 32×32, réparties en 10 classes équilibrées. Il est divisé en :

— 50 000 images d'entraînement,
— 10 000 images de test.

Pour le pré-entraînement SimCLR, nous utilisons uniquement les images d'entraînement, sans labels. Pour les stages supervisés, un split interne est réalisé afin de constituer un ensemble de validation.

CIFAR-10-C

CIFAR-10-C est un benchmark conçu pour évaluer la robustesse des modèles face aux corruptions. Il contient :

— 15 types de corruptions (bruit, flou, météo, compression, etc.),
— 5 niveaux de sévérité,
— 10 000 images par corruption et par sévérité.

Ce dataset est utilisé exclusivement pour le Stage C (TTT). Le pipeline télécharge automatiquement l'archive depuis Zenodo, la met en cache sur Google Drive et l'extrait localement.

3.3 Architecture du modèle

Le modèle utilisé est un **Vision Transformer Tiny (ViT-Tiny)**, adapté aux images de petite résolution. Ses caractéristiques principales sont :

— taille des patchs : 4×4,
— dimension d'embedding : 192,
— profondeur : 12 blocs Transformer,
— tête de projection SimCLR de dimension 128.

Le ViT est particulièrement adapté au SSL grâce à sa capacité d'attention globale et à sa compatibilité naturelle avec les augmentations fortes.

3.4 Stage A : Pré-entraînement SimCLR

Le pré-entraînement SimCLR constitue la première étape du pipeline. Il repose sur :

— des augmentations fortes (crop, jitter couleur, flou gaussien),
— une perte contrastive InfoNCE,
— un batch size SSL de 1024,
— un scheduler avec warmup puis décroissance cosinus.

L'objectif est de rapprocher les représentations de deux vues augmentées d'une même image tout en éloignant celles d'images différentes. Le modèle est entraîné pendant 200 époques, et le meilleur checkpoint est sauvegardé selon la loss de validation.

3.5 Stage B.1 : Linear Probe

Le linear probe consiste à :
— geler entièrement l'encodeur pré-entraîné,
— entraîner uniquement une couche linéaire sur les représentations,
— utiliser un batch size supervisé de 256.

Cette étape permet d'évaluer la qualité intrinsèque des représentations apprises par SimCLR. Elle sert également de point de comparaison avec le fine-tuning complet.

3.6 Stage B.2 : Fine-tuning

Le fine-tuning consiste à réentraîner l'ensemble du modèle (encodeur + tête) sur CIFAR-10 avec labels. Les hyperparamètres incluent :
— 30 époques d'entraînement,
— un learning rate plus faible que pour SimCLR,
— du label smoothing,
— un early stopping basé sur la loss de validation.

Cette étape permet d'adapter finement les représentations au dataset supervisé et conduit généralement à une précision supérieure à 90%.

3.7 Stage C : Test-Time Training

Le Test-Time Training (TTT) vise à adapter le modèle directement au moment du test, sans labels. Nous utilisons la méthode TENT, qui consiste à :
— geler toutes les couches sauf les normalisations,
— minimiser l'entropie des prédictions sur les images corrompues,
— adapter le modèle en ligne, batch par batch.

Cette adaptation locale permet d'améliorer la robustesse du modèle sur CIFAR-10-C, en particulier pour les corruptions sévères.

Chapitre 4

Implémentation

Ce chapitre décrit l'implémentation complète du pipeline développé dans ce projet. L'objectif est de présenter l'organisation du code, les classes principales, ainsi que les mécanismes assurant la modularité, la reproductibilité et la clarté du système. L'ensemble du code est structuré en modules cohérents, permettant d'exécuter chaque stage indépendamment tout en partageant une architecture commune.

4.1 Structure générale du code

Le projet est organisé selon une architecture modulaire inspirée des bonnes pratiques de recherche en vision par ordinateur. La structure générale est la suivante :

- `src/core/` : gestion de la configuration, pipeline d'exécution, gestion des checkpoints et du logging ;
- `src/models/` : définition du Vision Transformer et de la tête SimCLR ;
- `src/data/` : préparation des datasets, loaders SSL et supervisés, gestion de CIFAR-10-C ;
- `src/training/` : implémentation des différents trainers (SimCLR, Linear Probe, Fine-Tuning) ;
- `src/ttt/` : implémentation de l'adaptation en ligne TENT ;
- `configs/` : fichiers YAML définissant les hyperparamètres ;
- `notebooks/` : notebook Colab orchestrant l'exécution du pipeline.

Cette organisation permet de séparer clairement les responsabilités : modèles, données, entraînement, adaptation et orchestration.

4.2 BaseTrainer

La classe `BaseTrainer` constitue le cœur du système d'entraînement. Elle implémente un moteur générique comprenant :

- une boucle d'entraînement et de validation ;
- la gestion du scheduler et de l'optimiseur ;
- l'AMP (Automatic Mixed Precision) pour accélérer l'entraînement ;
- la journalisation des métriques (CSV + TensorBoard) ;
- la sauvegarde automatique des meilleurs checkpoints ;
- un mécanisme d'early stopping.

Les méthodes clés sont :

— `fit()` : boucle principale d'entraînement ;
— `_train_one_epoch()` : à implémenter par les classes filles ;
— `_validate()` : à implémenter par les classes filles.

Cette classe assure une interface commune pour tous les stages supervisés ou auto-supervisés.

4.3 SimCLRTrainer

La classe `SimCLRTrainer` hérite de `BaseTrainer` et implémente la logique spécifique à SimCLR :
— génération de deux vues augmentées pour chaque image ;
— passage dans l'encodeur ViT puis dans la tête de projection ;
— calcul de la perte InfoNCE ;
— normalisation des embeddings ;
— gestion des paires positives et négatives dans le batch.

La méthode `_train_one_epoch()` calcule la perte contrastive pour chaque batch, tandis que `_validate()` évalue la cohérence des représentations sur un ensemble de validation SSL.

Le meilleur checkpoint est sauvegardé sous le nom `simclr_best.pt`.

4.4 LinearProbeTrainer

Le `LinearProbeTrainer` implémente l'évaluation supervisée des représentations apprises. Ses caractéristiques principales sont :
— l'encodeur ViT est entièrement gelé ;
— seule une couche linéaire est entraînée ;
— la perte utilisée est l'entropie croisée ;
— les métriques incluent la précision top-1.

Cette étape permet d'évaluer la qualité intrinsèque des représentations produites par SimCLR, indépendamment du fine-tuning.

Le meilleur modèle est sauvegardé sous `linear_probe_best.pt`.

4.5 FineTuneTrainer

Le `FineTuneTrainer` réentraîne l'ensemble du modèle (encodeur + tête) sur CIFAR-10. Il reprend la structure du `BaseTrainer` mais avec :
— un learning rate plus faible ;
— un entraînement sur 30 époques ;
— du label smoothing ;
— un scheduler adapté ;
— une optimisation de toutes les couches.

Cette étape permet d'adapter finement les représentations au dataset supervisé. Le meilleur modèle est sauvegardé sous `finetune_best.pt`.

4.6 TestTimeAdapter (TENT)

Le module `TestTimeAdapter` implémente la méthode TENT (Test-Time Entropy Minimization). Son fonctionnement repose sur :

— le gel complet du modèle, sauf les couches de normalisation ;
— la minimisation de l'entropie des prédictions :

$$\mathcal{L}_{\text{TENT}} = -\sum_{c} p(c|x) \log p(c|x)$$

— une adaptation en ligne, batch par batch, sur les images corrompues ;
— l'absence totale de labels durant l'adaptation.

Cette approche permet d'améliorer la robustesse du modèle sur CIFAR-10-C sans réentraînement complet.

4.7 Notebook Colab et reproductibilité

Le notebook Colab constitue l'interface principale d'exécution du pipeline. Il assure :

— le chargement de la configuration YAML ;
— l'application d'overrides dynamiques ;
— l'exécution contrôlée des stages via des drapeaux ;
— la visualisation des métriques via TensorBoard ;
— la sauvegarde automatique des logs et checkpoints sur Google Drive.

Le notebook permet également de reprendre une expérience interrompue, grâce à la vérification systématique de la présence des artefacts nécessaires à chaque stage.

Cette organisation garantit une reproductibilité complète des expériences, même dans un environnement Colab éphémère.

Chapitre 5

Résultats

5.1 Convergence SimCLR

5.2 Linear Probe

5.3 Fine-tuning

5.4 CIFAR-10-C : baseline vs TTT

5.5 Analyse des gains de robustesse

Chapitre 6

Discussion

Les résultats obtenus au cours de ce projet permettent d'analyser de manière critique l'impact de l'apprentissage auto-supervisé et de l'adaptation en ligne sur la robustesse des modèles de vision. Cette discussion met en perspective les performances observées, les limites du pipeline proposé, ainsi que les enseignements tirés pour de futurs travaux.

Qualité des représentations apprises

Le pré-entraînement SimCLR a permis d'obtenir des représentations visuelles de haute qualité, comme en témoignent les performances du *linear probe* et du fine-tuning. Le modèle atteint plus de 90% de précision après fine-tuning, ce qui confirme l'efficacité du SSL pour initialiser un Vision Transformer même sur un dataset de petite taille comme CIFAR-10.

Ces résultats montrent que :

— les augmentations fortes jouent un rôle central dans la qualité des représentations ;
— le ViT-Tiny, malgré sa petite taille, bénéficie pleinement du pré-entraînement contrastif ;
— la tête de projection SimCLR est essentielle pour structurer l'espace latent.

Robustesse et adaptation en ligne

L'évaluation sur CIFAR-10-C met en évidence la sensibilité du modèle aux corruptions, même après fine-tuning supervisé. Cette observation est cohérente avec la littérature : les modèles entraînés sur des données propres ont tendance à se dégrader fortement en présence de bruit, de flou ou de perturbations de compression.

L'application de TENT améliore systématiquement les performances sur la majorité des corruptions. Cette amélioration confirme que :

— la minimisation de l'entropie constitue un signal d'adaptation efficace ;
— les couches de normalisation jouent un rôle clé dans la robustesse ;
— l'adaptation en ligne permet de corriger localement les décalages de distribution.

Cependant, les gains varient selon les corruptions. Certaines perturbations sévères (par exemple *impulse noise* ou *glass blur*) restent difficiles à corriger, ce qui suggère que TENT ne peut compenser qu'une partie du décalage de distribution.

Limites du pipeline

Bien que le pipeline soit fonctionnel et reproductible, plusieurs limites doivent être soulignées :

- **Taille du modèle** : ViT-Tiny reste un modèle léger, ce qui limite sa capacité d'apprentissage par rapport à des architectures plus larges.
- **Taille du dataset** : CIFAR-10 est un dataset relativement simple ; les conclusions pourraient différer sur des datasets plus complexes.
- **Adaptation locale** : TENT adapte uniquement les couches de normalisation, ce qui limite la capacité d'adaptation profonde du modèle.
- **Absence de régularisation spécifique à la robustesse** : aucune technique de type augmentation adversariale ou mixup n'a été intégrée.

Ces limites ouvrent la voie à des améliorations futures.

Enseignements et perspectives

Ce projet met en évidence plusieurs enseignements importants :

- l'apprentissage auto-supervisé constitue une alternative crédible à la supervision classique pour initialiser des modèles de vision ;
- les Vision Transformers bénéficient particulièrement du SSL grâce à leur architecture flexible ;
- l'adaptation en ligne est une solution simple mais efficace pour améliorer la robustesse ;
- la modularité du pipeline facilite l'exploration de nouvelles méthodes.

Des perspectives intéressantes incluent :

- l'utilisation de méthodes SSL plus récentes (MAE, DINO, iBOT) ;
- l'exploration de variantes de TTT plus puissantes (TTT++, EATA, SAR) ;
- l'évaluation sur des datasets plus complexes (ImageNet-C, Tiny-ImageNet-C) ;
- l'intégration de techniques de robustesse explicite (adversarial training, augmentations stochastiques).

Dans l'ensemble, ce travail montre que la combinaison SSL + ViT + TTT constitue une approche prometteuse pour améliorer la robustesse des modèles de vision face aux décalages de distribution.

Chapitre 7

Conclusion et perspectives

Ce travail avait pour objectif d'étudier l'apport de l'apprentissage auto-supervisé et de l'adaptation en ligne au moment du test pour améliorer la robustesse des modèles de vision face aux décalages de distribution. Pour cela, nous avons développé un pipeline complet reposant sur un Vision Transformer pré-entraîné avec SimCLR, évalué via *linear probe* et *fine-tuning*, puis adapté sur CIFAR-10-C à l'aide de la méthode TENT.

Les résultats obtenus montrent que l'apprentissage auto-supervisé constitue une stratégie efficace pour initialiser un modèle de vision. Le pré-entraînement SimCLR permet d'apprendre des représentations visuelles de haute qualité, comme en témoignent les performances élevées obtenues après fine-tuning supervisé. Le modèle atteint plus de 90% de précision sur CIFAR-10, confirmant la pertinence du SSL même sur des architectures compactes comme ViT-Tiny.

L'évaluation sur CIFAR-10-C met en évidence la sensibilité du modèle aux corruptions, un phénomène bien documenté dans la littérature. Toutefois, l'adaptation en ligne via TENT améliore systématiquement la robustesse du modèle, avec des gains significatifs sur la majorité des corruptions. Ces résultats confirment que la minimisation de l'entropie constitue un signal d'adaptation simple mais efficace pour réduire l'impact des décalages de distribution.

Au-delà des performances obtenues, ce projet a permis de concevoir un pipeline modulaire, reproductible et facilement extensible. L'utilisation de Google Colab, de configurations YAML, de checkpoints structurés et de logs détaillés garantit la transparence et la réplicabilité des expériences.

Perspectives

Plusieurs pistes d'amélioration peuvent être envisagées pour prolonger ce travail :

- **Explorer des méthodes SSL plus récentes** telles que MAE, DINO ou iBOT, qui ont démontré des performances supérieures à SimCLR sur des architectures Transformer.
- **Tester des variantes avancées de Test-Time Training** comme TTT++, EATA ou SAR, qui proposent des mécanismes d'adaptation plus stables et plus performants.
- **Évaluer le pipeline sur des datasets plus complexes** tels que Tiny-ImageNet, ImageNet-100 ou ImageNet-C, afin de mieux mesurer la scalabilité des approches étudiées.

— **Intégrer des techniques de robustesse explicite** comme l'adversarial training, les augmentations stochastiques ou les méthodes de calibration.
— **Étudier l'impact de la taille du modèle** en comparant ViT-Tiny avec des variantes plus larges (ViT-S, ViT-B), afin d'analyser le rôle de la capacité dans la robustesse.

En conclusion, ce projet montre que la combinaison de l'apprentissage auto-supervisé et de l'adaptation en ligne constitue une approche prometteuse pour améliorer la robustesse des modèles de vision. Les résultats obtenus ouvrent la voie à de nombreuses explorations futures, tant sur le plan méthodologique que sur le plan expérimental.

Bibliographie

Annexe A

Annexes

Cette section regroupe les éléments complémentaires permettant de documenter plus en détail l'implémentation, la configuration et la reproductibilité du pipeline. Les annexes incluent notamment la structure du dépôt, les fichiers de configuration, des extraits de code essentiels, ainsi que des informations sur l'environnement d'exécution.

A.1 Structure du dépôt

Le projet est organisé selon une architecture modulaire facilitant la lisibilité et la maintenance du code. La structure générale est la suivante :

```
|-- configs/
|   |-- simclr.yaml
|   |-- linear_probe.yaml
|   |-- finetune.yaml
|   `-- ttt.yaml
|-- src/
|   |-- core/
|   |-- data/
|   |-- models/
|   |-- training/
|   `-- ttt/
|-- notebooks/
|   `-- pipeline_colab.ipynb
|-- checkpoints/
|-- logs/
|-- readme.md
`-- requirements.txt
```

Cette organisation permet de séparer clairement les modules liés aux données, aux modèles, à l'entraînement et à l'adaptation.

A.2 Configuration YAML

Les fichiers YAML définissent les hyperparamètres utilisés dans chaque stage. Ils permettent de modifier facilement les paramètres sans toucher au code source. Voici un

exemple simplifié :

```
model:
name: vit_tiny
projection_dim: 128

training:
batch_size: 1024
epochs: 200
optimizer: adamw
lr: 3e-3
scheduler: cosine

data:
dataset: cifar10
augmentations: simclr
```

A.3 Extraits de code essentiels

Boucle d'entraînement générale

```
for epoch in range(self.epochs):
train_loss = self._train_one_epoch()
val_loss = self._validate()
self.logger.log(epoch, train_loss, val_loss)
self.checkpoint_manager.update(val_loss, model)
```

Perte InfoNCE utilisée dans SimCLR

```
loss = -log( exp(sim(z_i, z_j) / tau) /
sum_k exp(sim(z_i, z_k) / tau) )
```

A.4 Environnement d'exécution

Les expériences ont été réalisées sur Google Colab avec l'environnement suivant :

- GPU : NVIDIA T4 ou A100 selon disponibilité,
- Python 3.10,
- PyTorch 2.x,
- CUDA 11+,
- TensorBoard pour la visualisation.

L'utilisation de Google Drive permet de sauvegarder automatiquement les checkpoints et les logs afin de garantir la reproductibilité.

A.5 Commandes d'exécution du pipeline

Les différents stages peuvent être exécutés indépendamment via les drapeaux suivants dans le notebook Colab :

```
RUN_STAGE_A = True      # Pré-entraînement SimCLR
RUN_STAGE_B1 = True     # Linear Probe
RUN_STAGE_B2 = True     # Fine-tuning
RUN_STAGE_C = True      # Test-Time Training
```

A.6 Résultats TensorBoard

Les courbes d'entraînement (loss, accuracy, learning rate) sont accessibles via TensorBoard. Elles permettent de visualiser :

— la convergence de SimCLR,
— la progression du linear probe,
— l'amélioration du fine-tuning,
— la stabilité de TENT.

A.7 Reproductibilité

Pour reproduire l'ensemble des expériences :

1. Ouvrir le notebook Colab.
2. Monter Google Drive.
3. Télécharger CIFAR-10 et CIFAR-10-C.
4. Configurer les drapeaux d'exécution.
5. Lancer les stages dans l'ordre souhaité.

Les checkpoints sauvegardés permettent de reprendre une expérience à n'importe quel stage sans réentraînement complet.